1. EXIF metadata

```
Images/Brain-Channel.png
ResolutionUnit : 2
ImageDescription : LATERAL: Pivot to internal Domain Controller using Mimikatz.
YCbCrPositioning : 1
XResolution : 72.0
YResolution : 72.0
EXIF tags: 0
```

2. PNG text metadata

```
in/Lab6$ python3 lab6.py
Images found: 6
Images/Music.png
Copyright : RVhGSUw6IFR1bm5lbCBjb21wcmVzc2VkIGRhdGFiYXNlIGR1bXAgdmlhIEROUyBDb3ZlcnQgQ2hhbm5lbC4=
Make : Olympus
Model : OM-D E-M10
create-date : 2025-02-06T11:11:00
Images/Dopamine.png
XML:com.adobe.xmp : <?xpacket begin='' id='W5M0MpCehiHzreSzNTczkc9d'?>
<x:xmpmeta xmlns:x='adobe:ns:meta/' x:xmptk='Image::ExifTool 12.76'>
<rdf:RDF xmlns:rdf='http://www.w3.org/1999/02/22-rdf-syntax-ns#'>

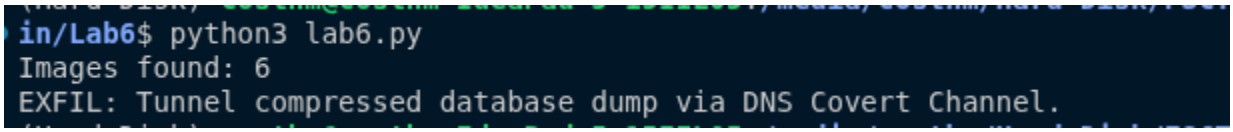
 <rdf:Description rdf:about=''
  xmlns:exif='http://ns.adobe.com/exif/1.0/'>
  <exif:MakerNote>PERSIST: Establish reverse shell via obfuscated cron job.</exif:MakerNote>
 </rdf:Description>
</rdf:RDF>
</x:xmpmeta>
<?xpacket end='r'?>
Images/Brain.png
Software : RECON: Perform DNS harvesting and Subdomain enumeration.
Images/Brain-Channel.png
Images/Chemicals.png
ImageDescription : EXPLOIT: Deploy SQL Injection payload to bypass WAF.
Images/Activation.png
Make : Nikon
Model : D750
Software : Adobe Lightroom 13.0
```

3. XMP metadata



4. Decode





5. Investigation

Image	Metadata type / field	Decoding needed?	Partial secret recovered
Activation.png	PNG Text, Image metadata	No	ASCIIVULN: Identify unpatched CVE-2024-XXXX in web gateway.
Brain-Channel.png	PNG Text, Image metadata	No	LATERAL: Pivot to internal Domain Controller using Mimikatz.HH
Brain.png	PNG Text, Image	No	RECON: Perform DNS

	metadata		harvesting and Subdomain enumeration.
Chemicals.png	PNG Text, Image metadata	No	ImageDescription: EXPLOIT: Deploy SQL Injection payload to bypass WAF.
Dopamine.png	XMP metadata	No	<exif:MakerNote>PERSIST: Establish reverse shell via obfuscated cron job.</exif:MakerNote>
Music.png	EXIF, PNG text image metadata	Yes	Tunnel compressed database dump via DNS covert channel

6. Reconstruct the complete secret

1. Stage: RECON Recovered message: Perform DNS harvesting and subdomain enumeration.
2. Stage: ASCIIVULN Recovered message: Identify unpatched CVE-2024-XXXX in web gateway.
3. Stage: EXPLOIT Recovered message: Deploy SQL Injection payload to bypass WAF.
4. Stage: LATERAL Recovered message: Pivot to internal Domain Controller using Mimikatz.
5. Stage: PERSIST Recovered message: Establish reverse shell via obfuscated cron job.
6. Stage: EXFIL Recovered message: Tunnel compressed database dump via DNS covert channel.

Explanation:
Recon gets information about the target's DNS, ASCIIVULN identifies a potential weakness in the web gateway from a known vulnerability, EXPLOIT tries to SQ: inject to bypass the web application firewall. LATERAL movement to access internal systems. PERSIST stage establishes a mechanism to maintain access with reverse shell though an obfuscated cron job. EXFIL finally transfers the compressed database dump out of the environment using DNS covert channel.

7. Metadata scanner

```
from pathlib import Path
from PIL import Image, ExifTags
import base64
import csv
import re

image_dir = Path("Images")
results = []

for image_path in sorted(image_dir.glob("*.png")):
    with Image.open(image_path) as img:
        metadata = {}

        # 1. EXIF metadata
        for tag_id, value in img.getexif().items():
            tag = ExifTags.TAGS.get(tag_id, tag_id)
            metadata[f"EXIF:{tag}"] = str(value)

        # 2. PNG text and other metadata
        for key, value in img.info.items():
            if key.lower() in ("xmp", "xml:com.adobe.xmp"):
                field = f"XMP:{key}"
            else:
                field = f"PNG:{key}"

            if isinstance(value, bytes):
                value = value.decode("utf-8", errors="replace")

            metadata[field] = str(value)

    suspected = []
    encodings = []
    reasons = []
    score = 0

    # 3. Inspect metadata values
    for field, value in metadata.items():
        # Look for labelled messages, without assuming their content
        if re.search(r"\b[A-Z][A-Z0-9_]*:", value):
            suspected.append(field)

        # Check whether the value may be Base64
        compact = "".join(value.split())

        if len(compact) >= 16:
            try:
                decoded = base64.b64decode(
```

```
                compact, validate=True
            ).decode("utf-8")

            if decoded.isprintable():
                encodings.append(field)

        except (ValueError, UnicodeDecodeError):
            pass

    # 4. Apply simple heuristic scoring
    if suspected:
        score += 10
        reasons.append("+10: Labelled message found in metadata")

    if encodings:
        score += 5
        reasons.append("+5: Readable Base64 candidate found")

    # Additional unusual metadata needs manual review.
    # Do not add another 5 just for the same encoded value.

    results.append({
        "filename": image_path.name,
        "format": img.format,
        "metadata_fields": ", ".join(metadata.keys()),
        "suspected_field": ", ".join(suspected),
        "encoding": (
            "Possible Base64: " + ", ".join(encodings)
            if encodings else "None identified"
        ),
        "risk_score": score,
        "reason": "; ".join(reasons) if reasons else
                "No indicators matched; manual review may still be needed."
    })

# 5. Export CSV
fields = [
    "filename", "format", "metadata_fields",
    "suspected_field", "encoding", "risk_score", "reason"
]

with open("metadata_report.csv", "w", newline="", encoding="utf-8-sig") as file:
    writer = csv.DictWriter(file, fieldnames=fields)
    writer.writeheader()
    writer.writerows(results)

print(f"Saved {len(results)} images to metadata_report.csv")
```

Reflection Questions:

1. Why can an image contain a hidden message even when `getexif()` returns no tags?
   An image can have hidden messages in other metadata formats like PNG text in description of comments, or XMP metadata. These are separated from EXIF so if getexif() does not return any tags when metadata have other information, hidden data can still be embedded in the image pixels using steganography.

2. How is **encoding** different from **encryption**? Is Base64 a secure way to hide information?
   Encoding converts data into another format so it can be stored or transmitted more easily. Encryption uses a cryptographic key to protect the information from unauthorized access. Based64 is an encoding method not encryption and does not provide security because anyone can decode it without a key, it only changes how the information is represented.

3. Why should investigators preserve the original files and record hashes?
   Investigator should preserve the original files to keep integrity of the evidence and avoiding writing over metadata. Recording cryptographic hashes provides a way to verify the files were not altered during examination or transfer. Helps with maintaining the chain of custody.

4. Can metadata alone prove that an image was edited or that an incident occurred? What corroborating evidence would you need?
   Metadata alone cannot prove that an image was edited or an incident occurred with it as metadata can be modified, removed, or falsified. Investigators should compare metadata with other evidence to support claims and establish a more reliable timeline.

5. How does this investigation connect to the automated evidence collection and analysis you practised in Labs 4 and 5?
   The investigation connects to other labs as using python to automate evidence collection, process multiple files, port scanning, http checker can help with identifying potentially useful information, and organize findings into a CSV file instead of manually reviewing log files. It help with establishing a timeline with evidence flow. Still, evidence collected from python scripts automatically should be manually reviewed and analysis before making a conclusion